

Comprehensive Test Plan and Test Cases
Algorithmic Trading System

Document Control
Document Title: Comprehensive Test Plan and Test Cases for Algorithmic Trading System
Version: 2.0
Date: April 24, 2025
Author: Senior Quality Assurance Engineer, Hedge Fund
Status: Draft
Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders
Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior QA Engineer	Initial draft of Test Plan and Test Cases
2.0	04/24/2025	Senior QA Engineer	Expanded test cases to include all possible types

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
QA Lead	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Objectives
 - 1.4. Assumptions and Constraints
 - 1.5. Definitions and Acronyms
- 2. [Test Plan Overview](#)
 - 2.1. Testing Objectives
 - 2.2. Scope of Testing
 - 2.3. Test Levels
 - 2.4. Test Types
 - 2.5. Test Environment
 - 2.6. Roles and Responsibilities
 - 2.7. Test Schedule
 - 2.8. Entry and Exit Criteria
 - 2.9. Test Deliverables
 - 2.10. Risks and Mitigation
- 3. [Test Cases](#)
 - 3.1. Unit Test Cases
 - 3.1.1. Data Ingestion Service
 - 3.1.2. Data Processing Service
 - 3.1.3. Data Storage Service
 - 3.1.4. Strategy Engine
 - 3.1.5. Order Execution Module
 - 3.1.6. Risk Management Module
 - 3.1.7. AI Agents Framework
 - 3.1.8. Reporting Module
 - 3.1.9. Monitoring and Logging Service
 - 3.2. Integration Test Cases
 - 3.2.1. Data Ingestion to Data Processing
 - 3.2.2. Data Processing to Strategy Engine
 - 3.2.3. Strategy Engine to Order Execution

- 3.2.4. Order Execution to Risk Management
- 3.2.5. AI Agents Framework Integration
- 3.2.6. API Gateway to Dashboard
- 3.2.7. Reporting Module to Data Storage
- 3.3. System Test Cases
 - 3.3.1. End-to-End Trading Workflow
 - 3.3.2. Market Watch Dashboard Functionality
 - 3.3.3. Backtesting Workflow
 - 3.3.4. Compliance and Audit Logging
- 3.4. Performance Test Cases
 - 3.4.1. Latency Testing
 - 3.4.2. Throughput Testing
 - 3.4.3. Scalability Testing
 - 3.4.4. Stress Testing
 - 3.4.5. Endurance Testing
 - 3.4.6. Spike Testing
- 3.5. Security Test Cases
 - 3.5.1. Authentication and Authorization
 - 3.5.2. Data Encryption
 - 3.5.3. Input Validation
 - 3.5.4. Penetration Testing
 - 3.5.5. Session Management
 - 3.5.6. Vulnerability Scanning
- 3.6. Usability Test Cases
 - 3.6.1. Dashboard Usability
 - 3.6.2. Accessibility Testing
 - 3.6.3. User Training Effectiveness
- 3.7. Compliance Test Cases
 - 3.7.1. Regulatory Compliance
 - 3.7.2. Audit Trail Integrity
 - 3.7.3. Data Retention Compliance
- 3.8. Regression Test Cases
 - 3.8.1. Core Functionality Regression
 - 3.8.2. Integration Regression
 - 3.8.3. Performance Regression
- 3.9. Acceptance Test Cases
 - 3.9.1. User Acceptance Testing
 - 3.9.2. Stakeholder Validation
- 3.10. Reliability Test Cases
 - 3.10.1. Fault Tolerance Testing
 - 3.10.2. Recovery Testing
 - 3.10.3. Availability Testing
- 3.11. Compatibility Test Cases
 - 3.11.1. Browser Compatibility
 - 3.11.2. Dependency Compatibility
- 3.12. Localization Test Cases
 - 3.12.1. Multi-Language Support
 - 3.12.2. Regional Data Formats
- 3.13. Installation and Configuration Test Cases
 - 3.13.1. Installation Testing
 - 3.13.2. Configuration Testing
- 3.14. Interoperability Test Cases
 - 3.14.1. IBKR API Interoperability
 - 3.14.2. Third-Party Tool Integration
- 3.15. Data Integrity Test Cases
 - 3.15.1. Data Consistency

- 3.15.2. Data Accuracy
 - 3.16. Boundary Test Cases
 - 3.16.1. Input Boundary Testing
 - 3.16.2. Performance Boundary Testing
 - 3.17. Negative Test Cases
 - 3.17.1. Invalid Input Handling
 - 3.17.2. Failure Scenario Handling
 - 4. [Test Execution and Reporting](#)
 - 4.1. Test Execution Process
 - 4.2. Defect Management
 - 4.3. Test Reporting
 - 5. [Missing Information and Best Practices](#)
 - 5.1. Missing Information
 - 5.2. Best Practices Applied
 - 6. [Glossary](#)
 - 7. [Appendices](#)
 - 7.1. Test Data Samples
 - 7.2. Test Tools and Frameworks
 - 7.3. References
-

1. Introduction

1.1. Purpose

The Comprehensive Test Plan and Test Cases document for the Algorithmic Trading System (ATS) provides an exhaustive strategy to validate the system's functionality, performance, security, usability, compliance, and reliability. This document includes all possible types of test cases to ensure the system is robust, secure, and compliant with regulatory standards. It serves as a guide for the Quality Assurance (QA) team to systematically verify the ATS, ensuring it meets the hedge fund's requirements for paper trading.

1.2. Scope

The ATS is a high-performance platform for algorithmic paper trading across stocks, indices, forex, and commodities on global exchanges (NYSE, NASDAQ, LSE, NSE). It integrates with the Interactive Brokers (IBKR) Paper Trading API and leverages Polars, DuckDB, and AI agents. The test scope includes:

- **Functional Testing:** Validate all components, including data ingestion, processing, storage, trading strategies, order execution, risk management, AI agents, dashboard, reporting, and monitoring.
- **Non-Functional Testing:** Verify performance (<100ms latency), scalability (10 users, 1TB data), security (encryption, authentication), reliability (99.9% uptime), usability (WCAG 2.1), and compatibility.
- **Compliance Testing:** Ensure adherence to SEC, GDPR, MiFID II, and SEBI regulations.
- **Integration Testing:** Test interactions between internal components and IBKR API.
- **System Testing:** Validate end-to-end workflows, including trading and backtesting.
- **Additional Testing:** Cover regression, acceptance, reliability, compatibility, localization, installation, interoperability, data integrity, boundary, and negative testing.
- **Test Environment:** Local deployment on Lenovo Legion Pro 5.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Cryptocurrency or non-traditional asset support.
- Mobile app testing (dashboard is web-based).
- Advanced AI model training validation.

1.3. Objectives

- Achieve 100% coverage of functional requirements.
- Ensure >80% code coverage for unit and integration tests.
- Validate end-to-end latency <100ms and throughput of 10,000 updates/second.
- Confirm 99.9% uptime with fault-tolerant mechanisms.
- Verify compliance with SEC, GDPR, MiFID II, and SEBI regulations.
- Ensure dashboard usability with <5 minutes training time.

- Mitigate all security vulnerabilities.
- Validate scalability for 10 concurrent users and 1TB historical data.
- Cover all edge cases, boundary conditions, and failure scenarios.

1.4. Assumptions and Constraints

Assumptions:

- IBKR Paper Trading API provides reliable mock data and execution.
- Test environment mirrors production (Lenovo Legion Pro 5).
- QA team is proficient in Python, pytest, Locust, OWASP ZAP, and accessibility tools.
- Traders provide feedback during usability testing.
- Budget supports 3-6 month testing phase.

Constraints:

- Timeline: 3-6 months, aligned with development.
- Hardware: Limited to Lenovo Legion Pro 5.
- IBKR API limits: 50 req/s (data), 100 msg/s (orders).
- Compliance: Must meet SEC, GDPR, MiFID II, SEBI standards.
- No cloud infrastructure; local deployment only.

1.5. Definitions and Acronyms

- **ATS**: Algorithmic Trading System.
- **IBKR**: Interactive Brokers.
- **OHLCV**: Open, High, Low, Close, Volume.
- **VW SMA**: Volume-Weighted Simple Moving Average.
- **MFI**: Money Flow Index.
- **ATR**: Average True Range.
- **Polars**: Data processing library.
- **DuckDB**: Analytical database.
- **pytest**: Python testing framework.
- **Locust**: Performance testing tool.
- **OWASP ZAP**: Security testing tool.
- **WCAG**: Web Content Accessibility Guidelines.

2. Test Plan Overview

2.1. Testing Objectives

- Validate functional correctness of all components and workflows.
- Ensure non-functional requirements (performance, security, usability, reliability) are met.
- Verify seamless integration with IBKR API and internal services.
- Confirm regulatory compliance through immutable audit trails.
- Achieve comprehensive test coverage, including edge cases and failure scenarios.
- Validate system stability under normal, stress, and failure conditions.
- Ensure user-friendly dashboard with minimal training time.

2.2. Scope of Testing

Components:

- Data Ingestion Service
- Data Processing Service
- Data Storage Service
- Strategy Engine
- Order Execution Module
- Risk Management Module
- AI Agents Framework
- Market Watch Dashboard
- Reporting Module
- Monitoring and Logging Service

Features:

- Real-time and historical market data processing.
- Trend-Following and Support and Resistance strategies.

- Order placement and execution in IBKR paper trading.
- Risk controls (position sizing, stop-loss, drawdown limits).
- AI-driven analysis and execution.
- Responsive dashboard with charts and trading controls.
- Performance, backtesting, and compliance reports.

2.3. Test Levels

- **Unit Testing:** Validate individual components (e.g., indicator calculations).
- **Integration Testing:** Test interactions between components and external systems.
- **System Testing:** Verify end-to-end workflows (e.g., trading, backtesting).
- **Acceptance Testing:** Confirm system meets user and regulatory requirements.

2.4. Test Types

- **Functional:** Validate features against requirements.
- **Non-Functional:**
 - Performance: Latency, throughput, scalability, stress, endurance, spike.
 - Security: Authentication, encryption, input validation, penetration, session management, vulnerability scanning.
 - Usability: Dashboard navigation, accessibility, training effectiveness.
 - Reliability: Fault tolerance, recovery, availability.
 - Compatibility: Browser, dependency compatibility.
 - Localization: Multi-language, regional formats.
- **Compliance:** Audit trails, data retention, regulatory adherence.
- **Regression:** Ensure no regressions after changes.
- **Acceptance:** Validate user and stakeholder requirements.
- **Installation/Configuration:** Verify setup and configuration.
- **Interoperability:** Test external system integration.
- **Data Integrity:** Ensure data consistency and accuracy.
- **Boundary:** Test input and performance limits.
- **Negative:** Validate error handling for invalid inputs and failures.

2.5. Test Environment

- **Hardware:** Lenovo Legion Pro 5 (NVIDIA RTX 3060, 32GB RAM).
- **OS:** Ubuntu 22.04.
- **Software:**
 - Python 3.10, FastAPI, ib_insync, Polars, DuckDB.
 - React 18, Tailwind CSS, Plotly.js.
 - RabbitMQ, Redis, Prometheus, Grafana.
 - Docker Compose for service orchestration.
- **Network:** Localhost for internal services, external IBKR API connection.
- **Data:** Mock OHLCV, trade logs, portfolio data from IBKR, synthetic datasets.

2.6. Roles and Responsibilities

Role	Responsibilities
QA Lead	Define test strategy, review results, manage schedule
QA Engineers	Design and execute test cases, report defects
Developers	Fix defects, support integration testing
Traders	Provide feedback for usability and acceptance testing
Compliance Officer	Validate compliance test cases
IT Administrator	Set up and maintain test environment

2.7. Test Schedule

Phase	Duration	Activities
Test Planning	2 weeks	Define test plan, identify test cases
Test Case Design	5 weeks	Write comprehensive test cases, prepare data
Unit Testing	4 weeks	Test components, achieve >80% coverage
Integration Testing	6 weeks	Test component interactions, IBKR integration

Phase	Duration	Activities
System Testing	4 weeks	Validate end-to-end workflows, backtesting
Performance Testing	4 weeks	Test latency, throughput, scalability, stress
Security Testing	3 weeks	Test authentication, encryption, vulnerabilities
Usability Testing	3 weeks	Validate dashboard with traders
Compliance Testing	2 weeks	Verify audit trails, regulatory adherence
Regression Testing	3 weeks	Ensure no regressions after fixes
Acceptance Testing	2 weeks	Final validation with stakeholders
Additional Testing	3 weeks	Compatibility, localization, reliability, etc.

Total Duration: ~28 weeks (7 months), overlapping with development.

2.8. Entry and Exit Criteria

Entry Criteria:

- Requirements finalized and approved.
- Test environment set up and stable.
- Test cases documented and reviewed.
- Mock and synthetic data prepared.
- Development code frozen for testing phase.

Exit Criteria:

- 100% functional test cases passed.
- 80% code coverage achieved.
- All critical/high-severity defects resolved.
- Performance targets met (<100ms latency, 10,000 updates/s).
- Security vulnerabilities mitigated.
- Compliance requirements validated.
- Usability feedback addressed.
- Stakeholder approval for deployment.

2.9. Test Deliverables

- Comprehensive Test Plan and Test Cases document.
- Test data sets (OHLCV, trade logs, portfolio, synthetic data).
- Test scripts (pytest, Locust, Selenium).
- Test execution reports.
- Defect reports with severity and status.
- Code coverage reports.
- Performance, security, and usability test results.
- Compliance validation report.
- User feedback summaries.

2.10. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
IBKR API downtime	Medium	High	Use mock data, cache historical data
Insufficient test data	Low	Medium	Generate synthetic OHLCV, use IBKR historical
Performance bottlenecks	Medium	High	Optimize Polars/DuckDB, profile with Locust
Security vulnerabilities	Medium	High	Regular OWASP ZAP scans, encrypt data
Compliance gaps	Low	High	Involve compliance officer early
Trader feedback delays	Medium	Medium	Schedule usability tests in advance
Test environment instability	Low	Medium	Use Docker Compose for consistency

3. Test Cases

3.1. Unit Test Cases

3.1.1. Data Ingestion Service

Objective: Validate data fetching from IBKR API.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ING-001	Fetch real-time OHLCV	IBKR connected	API Ticker: "AAPL", Timeframe: "1m"	OHLCV JSON	Data matches IBKR response
TC-ING-002	Fetch historical OHLCV	IBKR connected	API Ticker: "AAPL", Timeframe: "1d", Start: "2025-01-01"	Array of OHLCV JSON	Data matches historical records
TC-ING-003	Handle invalid ticker	IBKR connected	API Ticker: "INVALID"	Error: "invalid_ticker"	Error logged, no crash
TC-ING-004	Respect rate limit	IBKR connected	API 51 requests/s	Error: "rate_limit_exceeded"	Retry after 1s
TC-ING-005	Cache data in Redis	Redis running	Ticker: "AAPL", Timeframe: "1m"	Data stored with 60s TTL	Redis key exists
TC-ING-006	Reconnect on disconnect	IBKR disconnected	API Simulate disconnect	Reconnect within 5s	Connection restored
TC-ING-007	Handle empty response	IBKR connected	API No data for ticker	Log warning, skip processing	No crash
TC-ING-008	Validate data format	IBKR connected	API Malformed JSON	Log error, skip record	No crash

3.1.2. Data Processing Service

Objective: Validate data normalization and indicator calculations.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PROC-001	Normalize timestamps	Raw OHLCV	OHLCV with mixed timezones	UTC timestamps	All timestamps in UTC
TC-PROC-002	Clean outliers	Raw OHLCV	OHLCV with price $>3\sigma$	Outliers removed	Prices within 3σ
TC-PROC-003	Interpolate missing values	Raw OHLCV	OHLCV with missing close	Linear interpolation	No missing values
TC-PROC-004	Calculate SMA	VW Clean OHLCV	OHLCV for 55 periods	VW SMA value	Matches formula
TC-PROC-005	Calculate MFI	Clean OHLCV	OHLCV for 14 periods	MFI value	Matches formula
TC-PROC-006	Calculate ATR	Clean OHLCV	OHLCV for 21 periods	ATR value	Matches formula
TC-PROC-007	Handle corrupted data	Raw OHLCV	Negative volume	Skip record, log error	No crash
TC-PROC-008	Handle zero volume	zero Raw OHLCV	Volume: 0	Skip VW SMA calculation	No crash
TC-PROC-009	Validate indicator ranges	Clean OHLCV	MFI calculation	MFI between 0-100	Value within range

3.1.3. Data Storage Service

Objective: Validate data storage and retrieval.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-STOR-001	Store OHLCV	DuckDB running	OHLCV JSON	Data in ohlcv table	Query returns data
TC-STOR-002	Store trade log	DuckDB running	Trade JSON	Data in trade_log table	Query returns trade
TC-STOR-003	Store portfolio	DuckDB running	Portfolio JSON	Data in portfolio table	Query returns position
TC-STOR-004	Partition date/ticker by	DuckDB running	OHLCV for multiple dates	Partitioned files Parquet	Files exist
TC-STOR-005	Query performance	Data stored	SELECT * WHERE ticker="AAPL"	Results in <100ms	Query time <100ms
TC-STOR-006	Handle write failure	Disk full	OHLCV JSON	Retry 3 times, log error	No crash
TC-STOR-007	Validate schema	DuckDB running	Invalid JSON schema	Error: "schema_mismatch"	No data stored
TC-STOR-008	Handle concurrent writes	DuckDB running	Multiple writes OHLCV	All data stored	No data loss

3.1.4. Strategy Engine

Objective: Validate trading strategy logic.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-STRAT-001	Trend-Following signal	buy Processed OHLCV	MFI > VW SMA	Buy signal	Signal action: "BUY"
TC-STRAT-002	Trend-Following signal	sell Processed OHLCV	MFI < VW SMA	Sell signal	Signal action: "SELL"
TC-STRAT-003	Support/Resistance signal	buy Processed OHLCV	Price breaks R1	Buy signal	Signal action: "BUY"
TC-STRAT-004	Support/Resistance signal	sell Processed OHLCV	Price breaks S1	Sell signal	Signal action: "SELL"
TC-STRAT-005	Position sizing	Account £10,000	balance: Risk: 1%, ATR: 2.0	Quantity calculated	Matches formula
TC-STRAT-006	Handle invalid data	Corrupted OHLCV	Missing price	Skip signal, log error	No crash
TC-STRAT-007	No signal insufficient data	on Processed OHLCV	<14 periods	No signal	No signal generated

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-STRAT-008	Validate timestamp	signal Processed OHLCV	Buy signal	Signal timestamp matches data	Timestamp accurate

3.1.5. Order Execution Module

Objective: Validate order placement and monitoring.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ORD-001	Place order	market IBKR connected	Buy AAPL, 100 shares	Order "PENDING"	status: Order returned ID
TC-ORD-002	Place limit order	IBKR connected	Buy AAPL, price: 150.90	Order "PENDING"	status: Order returned ID
TC-ORD-003	Place order	bracket IBKR connected	Buy AAPL, stop-loss, take-profit	Order "PENDING"	status: Order returned ID
TC-ORD-004	Monitor status	order Order placed	Order ID	Status: "FILLED"	Status updated
TC-ORD-005	Handle order	invalid IBKR connected	Negative quantity	Error: "invalid_quantity"	No order placed
TC-ORD-006	Respect limit	rate IBKR connected	101 msg/s	Error: "rate_limit_exceeded"	Retry after 1s
TC-ORD-007	Cancel order	pending Order placed	Order ID	Status: "CANCELED"	Order removed
TC-ORD-008	Handle rejection	order IBKR connected	Insufficient funds	Error: "insufficient_funds"	No order placed

3.1.6. Risk Management Module

Objective: Validate risk controls.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-RISK-001	Validate size	position Account balance: £10,000	1% risk trade	Approved trade	Size matches formula
TC-RISK-002	Reject trade	high-risk Account balance: £10,000	3% risk trade	Error: "risk_violation"	Trade rejected
TC-RISK-003	Enforce stop-loss	Trade signal	Buy AAPL, stop-loss: $2 \times \text{ATR}$	Stop-loss set	Stop-loss price valid
TC-RISK-004	Trigger breaker	circuit Market data	Price change >5%	Trading halted	Alert logged
TC-RISK-005	Limit risk	portfolio Portfolio data	Open risk >10%	Error: "max_open_risk"	Trade rejected

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-RISK-006	Handle drawdown	Portfolio data	Drawdown >20%	Risk reduced to 0.5%	Risk parameters updated
TC-RISK-007	Validate parameters	risk Risk settings	Negative value	risk Error: "invalid_risk"	Parameters unchanged
TC-RISK-008	Handle missing portfolio data	No portfolio	Trade signal	Error: "missing_portfolio"	Trade rejected

3.1.7. AI Agents Framework

Objective: Validate AI-driven tasks.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-AI-001	Trade execution timing	Market data	OHLCV, volatility	Optimized trade price	Price within 0.1%
TC-AI-002	Risk adjustment	Portfolio data	High exposure	Reduced position size	Within risk limits
TC-AI-003	Market prediction	trend Historical OHLCV	14-day data	Uptrend prediction	Matches LSTM output
TC-AI-004	Handle API limits	rate IBKR connected	51 req/s	Throttled requests	No API errors
TC-AI-005	Log explanations	SHAP Trade decision	Buy signal	SHAP values logged	Explanations queryable
TC-AI-006	Fallback on failure	Agent failure	Invalid model output	Manual mode	No crash
TC-AI-007	Handle low confidence	Market data	Confidence <0.5	No trade recommendation	No signal generated
TC-AI-008	Validate model retraining	Historical data	New OHLCV	Updated weights	Model accuracy >80%

3.1.8. Reporting Module

Objective: Validate report generation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REP-001	Trade performance report	Trade data	Account ID	P&L, Sharpe ratio	Matches calculated values
TC-REP-002	Portfolio analytics	Portfolio data	Account ID	Exposure, VaR	Matches pandas calculations
TC-REP-003	Backtesting report	Historical data	Strategy ID, ticker	Total return, drawdown	Matches backtest results
TC-REP-004	Compliance report	Trade logs	Account ID	Immutable logs	Queryable logs
TC-REP-005	Export to CSV	Report data	Format: "csv"	CSV file	File downloadable
TC-REP-006	Handle query failure	DuckDB down	Report query	Retry 3 times, log error	No crash
TC-REP-007	Export to PDF	Report data	Format: "pdf"	PDF file	File downloadable

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REP-008	Handle large data	Trade data	1M trades	Report generated in <30s	No timeout

3.1.9. Monitoring and Logging Service

Objective: Validate metrics and logs.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-MON-001	Collect metrics	latency Prometheus running	API call	Latency <100ms	Metric in Prometheus
TC-MON-002	Visualize Grafana	in Grafana running	Metrics	Dashboard with latency	Graph displayed
TC-MON-003	Log API call	API request	GET /market-data	JSON log	Log in DuckDB
TC-MON-004	Alert on high latency	Prometheus running	Latency >100ms	Email/SMS alert	Alert received
TC-MON-005	Handle failure	logging DuckDB down	Log entry	Retry 3 times	No crash
TC-MON-006	Query historical logs	Logs stored	SELECT * WHERE level="ERROR"	Error logs	Query returns results
TC-MON-007	Log user actions	Dashboard running	User login	Action logged	Log in DuckDB
TC-MON-008	Handle high log volume	System running	10,000 logs/s	All logs stored	No data loss

3.2. Integration Test Cases

3.2.1. Data Ingestion to Data Processing

Objective: Validate data flow from ingestion to processing.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-001	Real-time data flow	IBKR, running	Redis AAPL OHLCV	Processed in DuckDB	OHLCV Processed within 100ms
TC-INT-002	Historical data flow	IBKR, running	Redis AAPL historical data	Processed in DuckDB	OHLCV All records processed
TC-INT-003	Error propagation	IBKR error	returns Invalid ticker	Error logged, crash	no Processing invalid data skips
TC-INT-004	High volume data	IBKR connected	100 scrips	All data processed	No data loss
TC-INT-005	Handle partial data	IBKR connected	Missing OHLCV fields	Partial processing, log warning	No crash

3.2.2. Data Processing to Strategy Engine

Objective: Validate indicator data to signal generation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-006	Trend-Following signal	Processed OHLCV	MFI > VW SMA	Buy signal via RabbitMQ	Signal received
TC-INT-007	Support/Resistance signal	Processed OHLCV	Price breaks R1	Buy signal via RabbitMQ	Signal received
TC-INT-008	Invalid data handling	Corrupted OHLCV	Missing price	No signal, error logged	No crash
TC-INT-009	Concurrent signals	Processed OHLCV	10 scrips	All signals generated	No signal loss
TC-INT-010	Signal prioritization	Processed OHLCV	Multiple signals	Highest priority signal processed	Correct signal executed

3.2.3. Strategy Engine to Order Execution

Objective: Validate signal to order placement.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-011	Execute buy signal	IBKR connected	Buy signal for AAPL	Order placed	Order returned ID
TC-INT-012	Execute sell signal	IBKR connected	Sell signal for AAPL	Order placed	Order returned ID
TC-INT-013	Handle invalid signal	Invalid signal	Negative quantity	Error logged	No order placed
TC-INT-014	Concurrent orders	IBKR connected	10 signals	All orders placed	No order loss
TC-INT-015	Order confirmation	IBKR connected	Order ID	Status: "FILLED"	Status updated

3.2.4. Order Execution to Risk Management

Objective: Validate risk checks before execution.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-016	Approve trade	valid Risk parameters set	1% risk trade	Order approved	Order placed
TC-INT-017	Reject trade	high-risk Risk parameters set	3% risk trade	Error: "risk_violation"	Order rejected
TC-INT-018	Trigger breaker	circuit Market data	Price change >5%	Trading halted	Alert logged
TC-INT-019	Concurrent checks	risk Risk parameters set	10 trades	All checked	No missed checks
TC-INT-020	Update parameters	risk Risk settings	New max risk: 2%	Parameters updated	Trades reflect new limits

3.2.5. AI Agents Framework Integration

Objective: Validate AI agent interactions.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-021	AI trade recommendation	Market data	OHLCV, volatility	Buy recommendation	Matches model output
TC-INT-022	AI risk adjustment	Portfolio data	High exposure	Reduced position	Within risk limits
TC-INT-023	Human override	AI recommendation	Override command	Manual execution	AI paused

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-024	Concurrent AI tasks	Market data	10 scrips	All tasks completed	No task failure
TC-INT-025	AI logging	Trade decision	Buy signal	SHAP values logged	Logs queryable

3.2.6. API Gateway to Dashboard

Objective: Validate API to dashboard communication.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-026	Real-time data display	API, dashboard running	AAPL OHLCV	Data in dashboard	Updates every 5-10s
TC-INT-027	Order placement	API, dashboard running	Buy AAPL	Order displayed	Status matches API
TC-INT-028	Handle API failure	API down	GET /market-data	Error message	Graceful degradation
TC-INT-029	Concurrent users	API, dashboard running	10 users	All data displayed	No lag
TC-INT-030	WebSocket updates	WebSocket running	AAPL OHLCV	Real-time updates	Updates every 5-10s

3.2.7. Reporting Module to Data Storage

Objective: Validate report generation from storage.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-031	Trade performance report	Trade data in DuckDB	Account ID	Report generated	Matches trade data
TC-INT-032	Compliance report	Trade logs	Account ID	Immutable logs	Queryable logs
TC-INT-033	Handle storage failure	DuckDB down	Report query	Retry, log error	No crash
TC-INT-034	Concurrent requests	report DuckDB running	10 report queries	All reports generated	No timeout
TC-INT-035	Validate accuracy	report Trade data	Account ID	Report matches data	No discrepancies

3.3. System Test Cases

3.3.1. End-to-End Trading Workflow

Objective: Validate complete trading process.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-001	Execute buy trade	System running	AAPL signal buy	Order filled, portfolio updated	Trade logged, P&L updated
TC-SYS-002	Execute sell trade	System running	AAPL signal sell	Order filled, portfolio updated	Trade logged, P&L updated
TC-SYS-003	Handle risk violation	System running	High-risk trade	Trade rejected	Error logged
TC-SYS-004	Circuit breaker activation	System running	Price change >5%	Trading halted	Alert logged
TC-SYS-005	Concurrent trades	System running	10 trades	All executed	No failures

3.3.2. Market Watch Dashboard Functionality

Objective: Validate dashboard features.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-006	Display real-time data	Dashboard running	AAPL OHLCV	Data in table	Updates every 5-10s
TC-SYS-007	Render candlestick chart	Dashboard running	AAPL OHLCV	Chart with VW SMA, MFI	Renders in <1s
TC-SYS-008	Place order via dashboard	Dashboard running	Buy AAPL	Order status displayed	Order ID matches API
TC-SYS-009	Manage watchlist	Dashboard running	Add AAPL	AAPL in watchlist	Watchlist updated
TC-SYS-010	Handle dashboard crash	Dashboard running	Simulate crash	Error message, reload option	No data loss

3.3.3. Backtesting Workflow

Objective: Validate strategy backtesting.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-011	Backtest Following	Trend- Historical data	AAPL, 2020-2025	Report with returns, Sharpe	Matches expected metrics
TC-SYS-012	Backtest Support/Resistance	Historical data	AAPL, 2020-2025	Report with drawdown	Matches expected metrics
TC-SYS-013	Handle missing data	Historical data	Partial OHLCV	Error logged, partial results	No crash
TC-SYS-014	Backtest with large data	Historical data	1TB data	Report in <60s	No timeout

3.3.4. Compliance and Audit Logging

Objective: Validate regulatory compliance.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-015	Log trade activity	System running	Buy AAPL	Immutable log	Log queryable
TC-SYS-016	Retain logs for 7 years	7 Logs stored	Trade logs	Logs accessible	No deletion
TC-SYS-017	Detect market abuse	Trade data	Spoofing pattern	Alert logged	Compliance notified
TC-SYS-018	Log user access	System running	User login	Access logged	Log queryable

3.4. Performance Test Cases

3.4.1. Latency Testing

Objective: Validate system latency.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-001	End-to-end latency	System running	AAPL signal	buy Latency <100ms	Measured via Prometheus
TC-PERF-002	Indicator calculation	OHLCV data	100 scrips	<1s	Measured via Locust
TC-PERF-003	Dashboard refresh	Dashboard running	AAPL OHLCV	Every 5-10s	No lag observed
TC-PERF-004	API response time	API running	GET /market-data	<50ms	Measured via Locust

3.4.2. Throughput Testing

Objective: Validate system throughput.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-005		Data updates	System running	10,000 updates/s	All processed	No dropped updates
TC-PERF-006		Trade execution	System running	1,000 trades/min	All executed	No failures
TC-PERF-007		Report generation	Data stored	1 year of data	<10s	Report generated
TC-PERF-008		WebSocket updates	WebSocket running	100 scrips	10,000 msg/s	No message loss

3.4.3. Scalability Testing

Objective: Validate system scalability.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-009		Concurrent users	System running	10 users	No degradation	Latency <100ms
TC-PERF-010		Historical data	Data stored	1TB data	Queries <100ms	No timeout
TC-PERF-011		High load	System running	100 scrips, 10 users	Stable performance	No crashes
TC-PERF-012		Scale to 20 users	System running	20 users	Graceful degradation	Latency <200ms

3.4.4. Stress Testing

Objective: Validate system under extreme conditions.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-013		Max data volume	System running	10TB data	Stable queries	No crashes
TC-PERF-014		Max concurrent users	System running	50 users	Graceful degradation	No data loss
TC-PERF-015		Max trade volume	System running	10,000 trades/min	Throttled execution	No crashes
TC-PERF-016		Low memory	System running	1GB RAM	Reduced performance	No crashes

3.4.5. Endurance Testing

Objective: Validate system stability over time.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-017		24-hour operation	System running	Normal load	Stable performance	No crashes
TC-PERF-018		7-day operation	System running	Continuous trades	No memory leaks	Stable metrics
TC-PERF-019		Log accumulation	System running	1M logs/day	Logs stored	No storage issues
TC-PERF-020		Data growth	System running	1TB data/month	Queries <100ms	No slowdown

3.4.6. Spike Testing

Objective: Validate system under sudden load spikes.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-021	Sudden spike	trade	System running	5,000 trades in 1s	Throttled execution	No crashes
TC-PERF-022	Sudden spike	data	System running	100,000 updates in 1s	Queued processing	No data loss
TC-PERF-023	User spike		System running	50 users in 1s	Graceful degradation	No crashes
TC-PERF-024	API request spike		API running	1,000 req/s	Rate limiting	429 responses

3.5. Security Test Cases

3.5.1. Authentication and Authorization

Objective: Validate secure access.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-001	Valid token	JWT	Keycloak running	Valid token	Access granted	API succeeds call
TC-SEC-002	Invalid token		Keycloak running	Invalid token	401 Unauthorized	Access denied
TC-SEC-003	MFA for orders		Keycloak running	Order without MFA	401 Unauthorized	Access denied
TC-SEC-004	Role-based access		Keycloak running	Trader accesses admin endpoint	403 Forbidden	Access denied
TC-SEC-005	Token expiration		Keycloak running	Expired token	401 Unauthorized	Access denied

3.5.2. Data Encryption

Objective: Validate data security.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-006	TLS 1.3 for API		Nginx running	API call	Encrypted traffic	Wireshark shows TLS 1.3
TC-SEC-007	AES-256 storage	for	DuckDB running	OHLCV data	Encrypted Parquet	Unreadable without key
TC-SEC-008	Credential encryption		Vault running	API credentials	Encrypted vault	No exposure in plaintext
TC-SEC-009	WebSocket encryption		WebSocket running	Market data	Encrypted traffic	Wireshark shows TLS 1.3

3.5.3. Input Validation

Objective: Validate input handling.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-010	Prevent injection	SQL API running	Malicious query	400 Bad Request	No database compromise
TC-SEC-011	Prevent XSS	Dashboard running	Script in input	Sanitized output	No script execution
TC-SEC-012	Validate quantity	order API running	Negative quantity	422 Unprocessable Entity	Order rejected
TC-SEC-013	Prevent overflow	buffer API running	Large input string	400 Bad Request	No crash

3.5.4. Penetration Testing

Objective: Identify exploitable vulnerabilities.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-014	Brute force login	Keycloak running	1,000 login attempts	Account locked	No unauthorized access
TC-SEC-015	API endpoint exposure	API running	Unauthenticated request	401 Unauthorized	No data leakage
TC-SEC-016	Session hijacking	Dashboard running	Stolen session ID	Session invalidated	No access
TC-SEC-017	Data exfiltration	System running	Malicious query	No sensitive data returned	Data protected

3.5.5. Session Management

Objective: Validate session security.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-018	Session timeout	Dashboard running	Idle for 30min	Session expired	User logged out
TC-SEC-019	Concurrent sessions	Dashboard running	Login from 2 devices	Previous session invalidated	Only one active session
TC-SEC-020	Secure cookie	Dashboard running	Login request	Cookie with HttpOnly flags	Secure, Flags present
TC-SEC-021	Session revocation	Keycloak running	Revoke token	All sessions terminated	No access

3.5.6. Vulnerability Scanning

Objective: Identify security weaknesses.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-022	OWASP ZAP scan	System running	Full scan	No critical vulnerabilities	All issues mitigated
TC-SEC-023	Dependency vulnerabilities	System running	Scan with Dependabot	No known vulnerabilities	All dependencies patched
TC-SEC-024	Misconfiguration check	Nginx running	Scan for open ports	Only port 8000 open	No unauthorized access
TC-SEC-025	CSRF protection	Dashboard running	Form submission	CSRF token required	No unauthorized submission

3.6. Usability Test Cases

3.6.1. Dashboard Usability

Objective: Validate user experience.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-001	Learn navigation	Dashboard running	New user	<5min to navigate	User completes tasks
TC-USE-002	Place order	Dashboard running	Buy AAPL	Order placed	Order confirmed in <10s
TC-USE-003	View chart	Dashboard running	AAPL OHLCV	Chart renders	Visible in <1s
TC-USE-004	Customize dashboard	Dashboard running	Rearrange widgets	Layout saved	Layout persists

3.6.2. Accessibility Testing

Objective: Validate WCAG 2.1 compliance.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-005	Keyboard navigation	Dashboard running	Navigate keyboard	via All elements accessible	No stuck focus
TC-USE-006	High contrast mode	Dashboard running	Enable contrast	high Readable text	Contrast ratio >4.5:1
TC-USE-007	Screen reader support	Dashboard running	Use NVDA	ARIA labels read	All elements described
TC-USE-008	Color blindness	Dashboard running	Simulate deuteranopia	Distinguishable elements	No confusion

3.6.3. User Training Effectiveness

Objective: Validate training efficiency.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-009	Training time	Dashboard running	New user, 5min training	User places order	Order placed correctly
TC-USE-010	Error comprehension	Dashboard running	Error message	User understands issue	User corrects input
TC-USE-011	Help documentation	Dashboard running	Access help	Relevant guide displayed	User completes task
TC-USE-012	Trader feedback	Dashboard running	Trader dashboard uses	Positive feedback	>80% satisfaction

3.7. Compliance Test Cases

3.7.1. Regulatory Compliance

Objective: Validate regulatory adherence.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMP-001	Log all trades	System running	Buy AAPL	Immutable log	Log queryable
TC-COMP-002	Monitor market abuse	Trade data	Spoofing pattern	Alert logged	Compliance notified
TC-COMP-003	GDPR data protection	User data	User request to delete	Data anonymized	No PII retained
TC-COMP-004	MiFID II reporting	Trade data	Trade execution	Transaction report	Report generated

3.7.2. Audit Trail Integrity

Objective: Validate audit trail immutability.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMP-005	Prevent log modification	Logs stored	Attempt to edit log	Error: "immutable"	Log unchanged
TC-COMP-006	Query audit trail	Logs stored	SELECT * WHERE user_id="user123"	All user actions	Logs returned
TC-COMP-007	Log API access	API running	GET /market-data	Access logged	Log in DuckDB
TC-COMP-008	Log integrity check	Logs stored	Checksum validation	Checksum matches	No tampering detected

3.7.3. Data Retention Compliance

Objective: Validate data retention policies.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMP-009	Retain logs for 7 years	Logs stored	Trade logs	Logs accessible	No deletion
TC-COMP-010	Delete expired user data	User data	Data >7 years	Data deleted	No retention violation
TC-COMP-011	Backup logs	Logs stored	Backup request	Logs backed up	Backup queryable
TC-COMP-012	Restore logs	Backup stored	Restore request	Logs restored	No data loss

3.8. Regression Test Cases

3.8.1. Core Functionality Regression

Objective: Ensure no regressions in core features.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REG-001	Data ingestion	System running	AAPL OHLCV	Data processed	Matches previous behavior
TC-REG-002	Order execution	System running	Buy AAPL	Order placed	Matches previous behavior
TC-REG-003	Strategy signals	System running	AAPL OHLCV	Buy signal	Matches previous behavior
TC-REG-004	Dashboard display	Dashboard running	AAPL OHLCV	Data displayed	Matches previous behavior

3.8.2. Integration Regression

Objective: Ensure no regressions in integrations.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REG-005	IBKR integration	API IBKR connected	AAPL OHLCV	Data fetched	Matches previous behavior
TC-REG-006	Strategy execution	to System running	Buy signal	Order placed	Matches previous behavior
TC-REG-007	API to dashboard	API, dashboard running	AAPL OHLCV	Data displayed	Matches previous behavior
TC-REG-008	Reporting integration	Data stored	Report query	Report generated	Matches previous behavior

3.8.3. Performance Regression

Objective: Ensure no performance degradation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REG-009	End-to-end latency	System running	AAPL buy signal	Latency <100ms	Matches previous metrics
TC-REG-010	Throughput	System running	10,000 updates/s	All processed	Matches previous metrics
TC-REG-011	Query performance	Data stored	SELECT * WHERE ticker="AAPL"	<100ms	Matches previous metrics
TC-REG-012	Dashboard refresh	Dashboard running	AAPL OHLCV	Every 5-10s	Matches previous metrics

3.9. Acceptance Test Cases

3.9.1. User Acceptance Testing

Objective: Validate system meets user needs.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ACC-001	Trader places order	System running	Buy AAPL	Order placed	Trader confirms success
TC-ACC-002	View portfolio	System running	Portfolio request	Positions displayed	Trader confirms accuracy
TC-ACC-003	Generate report	System running	Performance report	Report generated	Trader confirms usability
TC-ACC-004	Monitor watchlist	System running	Add AAPL	Watchlist updated	Trader confirms functionality

3.9.2. Stakeholder Validation

Objective: Validate system meets business goals.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ACC-005	Compliance report	System running	Compliance query	Immutable logs	Stakeholder approves
TC-ACC-006	Performance metrics	System running	Performance report	Sharpe ratio, P&L	Stakeholder approves
TC-ACC-007	Scalability	System running	10 users	Stable performance	Stakeholder approves
TC-ACC-008	Security audit	System running	Security scan	No vulnerabilities	Stakeholder approves

3.10. Reliability Test Cases

3.10.1. Fault Tolerance Testing

Objective: Validate system handles failures.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REL-001	Handle IBKR API failure	System running	Simulate downtime	API Use cached data	No crash
TC-REL-002	Handle DuckDB failure	System running	Simulate DB crash	Retry 3 times	No data loss
TC-REL-003	Handle RabbitMQ failure	System running	Simulate crash	queue Queue messages	No message loss
TC-REL-004	Handle partial data	System running	Missing fields	OHLCV Process available data	No crash

3.10.2. Recovery Testing

Objective: Validate system recovery.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REL-005	Recover from crash	System running	Simulate crash	service Service restarts	No data loss
TC-REL-006	Recover from network failure	System running	Disconnect network	Reconnect within 5s	No data loss
TC-REL-007	Recover from power failure	System running	Simulate loss	power System restarts	No corruption
TC-REL-008	Recover logs	Logs stored	Simulate log loss	Restore backup	from Logs queryable

3.10.3. Availability Testing

Objective: Validate system uptime.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REL-009	99.9% uptime	System running	30-day test	<0.1% downtime	Metrics confirm uptime
TC-REL-010	Handle service restart	System running	Restart service	<10s downtime	No data loss
TC-REL-011	Handle rolling updates	System running	Deploy update	Zero downtime	No interruptions
TC-REL-012	Monitor availability	Prometheus running	Uptime metrics	Alerts on downtime	Alerts received

3.11. Compatibility Test Cases

3.11.1. Browser Compatibility

Objective: Validate dashboard across browsers.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMPAT-001	Chrome compatibility	Dashboard running	Chrome latest	Full functionality	No rendering issues
TC-COMPAT-002	Firefox compatibility	Dashboard running	Firefox latest	Full functionality	No rendering issues
TC-COMPAT-003	Safari compatibility	Dashboard running	Safari latest	Full functionality	No rendering issues
TC-COMPAT-004	Edge compatibility	Dashboard running	Edge latest	Full functionality	No rendering issues

3.11.2. Dependency Compatibility

Objective: Validate dependency versions.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMPAT-005	Python compatibility	3.10 System running	Python 3.10	Full functionality	No errors
TC-COMPAT-006	Polars version	System running	Latest Polars	Data processing works	No errors
TC-COMPAT-007	DuckDB version	System running	Latest DuckDB	Queries work	No errors
TC-COMPAT-008	React compatibility	18 Dashboard running	React 18	Dashboard renders	No errors

3.12. Localization Test Cases

3.12.1. Multi-Language Support

Objective: Validate language support.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-LOC-001	English UI	Dashboard running	Select English	English text	All text in English
TC-LOC-002	Hindi UI	Dashboard running	Select Hindi	Hindi text	All text in Hindi
TC-LOC-003	French UI	Dashboard running	Select French	French text	All text in French
TC-LOC-004	Missing translations	Dashboard running	Select unsupported language	Fallback to English	No blank text

3.12.2. Regional Data Formats

Objective: Validate regional formats.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-LOC-005	US date format	Dashboard running	Select locale	US MM/DD/YYYY	Dates in US format
TC-LOC-006	UK date format	Dashboard running	Select locale	UK DD/MM/YYYY	Dates in UK format
TC-LOC-007	Currency format	Dashboard running	Select INR	₹1,00,000	Currency in INR format
TC-LOC-008	Decimal separator	Dashboard running	Select locale	EU 1.000,00	Comma separator used

3.13. Installation and Configuration Test Cases

3.13.1. Installation Testing

Objective: Validate system installation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INST-001	Docker Compose install	Ubuntu 22.04	Run docker-compose up	All services running	No errors
TC-INST-002	Dependency installation	Ubuntu 22.04	Run poetry install	All dependencies installed	No errors
TC-INST-003	Missing dependencies	Ubuntu 22.04	Omit Python	Installation fails	Error message
TC-INST-004	Clean install	Fresh Ubuntu	Install from scratch	System running	No errors

3.13.2. Configuration Testing

Objective: Validate system configuration.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INST-005	Valid config	System installed	Valid .env file	System starts	No errors
TC-INST-006	Invalid config	System installed	Invalid API key	System fails to start	Error logged
TC-INST-007	Environment variables	System installed	Set PORT=8001	System runs on 8001	Port correct
TC-INST-008	Default config	System installed	No .env file	Default settings used	System starts

3.14. Interoperability Test Cases

3.14.1. IBKR API Interoperability

Objective: Validate IBKR API integration.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INTER-001	Fetch OHLCV	IBKR connected	AAPL, 1m	OHLCV data	Matches IBKR response
TC-INTER-002	Place order	IBKR connected	Buy AAPL	Order placed	Order ID returned
TC-INTER-003	Handle changes	API IBKR connected	Updated version	API System adapts	No errors
TC-INTER-004	Rate limit handling	limit IBKR connected	51 req/s	Throttled requests	No API errors

3.14.2. Third-Party Tool Integration

Objective: Validate third-party tool compatibility.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INTER-005		Prometheus integration	Prometheus running	System metrics	Metrics collected	Metrics in Prometheus
TC-INTER-006		Grafana integration	Grafana running	Metrics	Dashboard displayed	Graphs visible
TC-INTER-007		RabbitMQ integration	RabbitMQ running	Trade signal	Signal queued	Signal received
TC-INTER-008		Redis integration	Redis running	OHLCV data	Data cached	Data retrievable

3.15. Data Integrity Test Cases

3.15.1. Data Consistency

Objective: Validate data consistency across components.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-DATA-001		OHLCV consistency	System running	AAPL OHLCV	Same data in Redis, DuckDB	No discrepancies
TC-DATA-002		Trade consistency	log System running	Buy AAPL	Same trade in logs, portfolio	No discrepancies
TC-DATA-003		Portfolio consistency	System running	Buy AAPL	Portfolio matches trades	No discrepancies
TC-DATA-004		Concurrent updates	System running	10 trades	All updates consistent	No data conflicts

3.15.2. Data Accuracy

Objective: Validate data accuracy.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-DATA-005		OHLCV accuracy	System running	AAPL OHLCV	Matches IBKR data	No deviations
TC-DATA-006		P&L accuracy	System running	Buy AAPL, sell AAPL	P&L matches trades	No deviations
TC-DATA-007		Indicator accuracy	System running	AAPL OHLCV	VW SMA matches formula	No deviations
TC-DATA-008		Report accuracy	System running	Performance report	Matches trade data	No deviations

3.16. Boundary Test Cases

3.16.1. Input Boundary Testing

Objective: Validate input boundaries.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-BND-001		Max quantity	System running	1M shares	Error: "max_quantity_exceeded"	Order rejected
TC-BND-002		Min quantity	System running	0 shares	Error: "invalid_quantity"	Order rejected
TC-BND-003		Max ticker length	System running	100-char ticker	Error: "invalid_ticker"	No processing
TC-BND-004		Max timeframe	System running	1y timeframe	Error: "invalid_timeframe"	No processing

3.16.2. Performance Boundary Testing

Objective: Validate performance limits.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-BND-005	Max volume	data	System running	10TB data	Stable queries	No crashes
TC-BND-006	Max users		System running	50 users	Graceful degradation	No crashes
TC-BND-007	Max trades		System running	10,000 trades/min	Throttled execution	No crashes
TC-BND-008	Min resources		System running	1GB RAM	Reduced performance	No crashes

3.17. Negative Test Cases

3.17.1. Invalid Input Handling

Objective: Validate handling of invalid inputs.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-NEG-001	Invalid ticker		System running	Ticker: "INVALID"	Error: "invalid_ticker"	No processing
TC-NEG-002	Negative price		System running	Price: -100	Error: "invalid_price"	No processing
TC-NEG-003	Malformed JSON		API running	Invalid JSON	400 Bad Request	No crash
TC-NEG-004	Invalid timeframe		System running	Timeframe: "1w"	Error: "invalid_timeframe"	No processing

3.17.2. Failure Scenario Handling

Objective: Validate handling of failures.

Test ID	Case	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-NEG-005	IBKR downtime	API	System running	Simulate failure	API Use cached data	No crash
TC-NEG-006	DuckDB crash		System running	Simulate crash	DB Retry 3 times	No data loss
TC-NEG-007	Network failure		System running	Disconnect network	Reconnect within 5s	No data loss
TC-NEG-008	Insufficient funds		System running	Trade > balance	Error: "insufficient_funds"	No order placed

4. Test Execution and Reporting

4.1. Test Execution Process

1. **Setup:** Configure test environment, load test data.
2. **Execution:**
 - o Run unit tests during development.
 - o Execute integration tests after component completion.
 - o Perform system, performance, security, usability, and other tests in staging.
 - o Conduct regression tests after defect fixes.
 - o Execute acceptance tests with stakeholders.
3. **Logging:** Record results in JIRA with pass/fail status.
4. **Defect Reporting:** Log defects in JIRA with severity and steps to reproduce.
5. **Review:** QA Lead reviews results, escalates critical issues.

4.2. Defect Management

- **Severity Levels:**
 - o Critical: System crash, data loss, security breach.
 - o High: Major functionality failure, compliance violation.

- Medium: Minor functionality issue, usability flaw.
 - Low: Cosmetic issues.
- **Process:**
 - Log in JIRA with description, severity, screenshots.
 - Assign to developers for resolution.
 - Retest after fix, update status.
 - Escalate unresolved critical defects to stakeholders.
- **Resolution Time:**
 - Critical: <24 hours.
 - High: <48 hours.
 - Medium: <1 week.
 - Low: <2 weeks.

4.3. Test Reporting

- **Daily Reports:** Test execution status, defects found/resolved.
- **Weekly Reports:** Progress against schedule, coverage metrics.
- **Final Report:** Summary of test results, defects, readiness for deployment.
- **Tools:** JIRA for tracking, pytest for coverage, Grafana for performance metrics.

5. Missing Information and Best Practices

5.1. Missing Information

- Exact IBKR API mock data specifications.
- Detailed compliance requirements for paper trading.
- Specific trader personas for usability testing.
- Performance benchmarks for Lenovo Legion Pro 5.
- News feed integration details.

5.2. Best Practices Applied

- **Mock Data:** Used synthetic OHLCV based on market patterns. **Reason:** Ensures comprehensive test coverage.
- **Compliance:** Implemented 7-year log retention and immutable audit trails. **Reason:** Aligns with SEC/GDPR standards.
- **Usability:** Assumed intermediate traders for testing. **Reason:** Matches hedge fund user base.
- **Performance:** Targeted <100ms latency based on high-frequency trading standards. **Reason:** Ensures competitiveness.
- **Security:** Adopted OWASP ZAP, TLS 1.3, and penetration testing. **Reason:** Industry-standard for secure systems.
- **Comprehensive Testing:** Included all test types (e.g., stress, spike, localization). **Reason:** Ensures robustness across scenarios.

6. Glossary

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **pytest:** Python testing framework.
- **Locust:** Performance testing tool.
- **OWASP ZAP:** Security testing tool.

7. Appendices

7.1. Test Data Samples

OHLCV:

```
{
  "ticker": "AAPL",
  "exchange": "NASDAQ",
```

```
"timestamp": "2025-01-01T10:00:00Z",  
"open": 150.25,  
"high": 151.10,  
"low": 149.80,  
"close": 150.90,  
"volume": 1000000  
}
```

Trade Log:

```
{  
  "trade_id": "T12345",  
  "timestamp": "2025-01-01T10:00:00Z",  
  "ticker": "AAPL",  
  "strategy": "trend_following",  
  "action": "BUY",  
  "quantity": 100,  
  "price": 150.90,  
  "status": "FILLED"  
}
```

7.2. Test Tools and Frameworks

- **pytest**: Unit and integration testing.
- **Locust**: Performance, stress, spike testing.
- **OWASP ZAP**: Security and penetration testing.
- **Selenium**: Browser and usability testing.
- **Postman**: API testing.
- **NVDA**: Accessibility testing.
- **Prometheus/Grafana**: Performance and availability monitoring.
- **JIRA**: Defect tracking and reporting.
- **Dependabot**: Dependency vulnerability scanning.

7.3. References

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
 - Polars: <https://pola.rs/>
 - DuckDB: <https://duckdb.org/>
 - pytest: <https://docs.pytest.org/>
 - OWASP ZAP: <https://owasp.org/www-project-zap/>
 - WCAG 2.1: <https://www.w3.org/TR/WCAG21/>
-